BRICK BREAKER GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive Brick Breaker game using Python Tkinter that allows players to control a paddle using keyboard arrow keys or 'A' and 'D' keys, bounce a ball, break colored bricks, increase their score, and clear the board while avoiding letting the ball drop below the paddle. The game includes a graphical interface, score tracking, win/game- over detection, and restart and exit options.

PROCEDURE

1. Start the program and import the required Python library: tkinter.
2. Create the main game window using Tkinter and set the title as "BRICK BREAKER".
3. Design a graphical game board using the Canvas widget with dimensions 600x450 and background colors. 4. Create the paddle and ball at their initial starting positions on the canvas.
5. Generate five distinct horizontal rows of multi-colored bricks on the upper portion of the game board.
6. Implement keyboard controls using the Left/Right arrow keys or 'A'/'D' keys to move the paddle horizontally.
7. Animate the ball continuously using Tkinter's after() method to update its spatial trajectory.
8. Implement collision physics to bounce the ball off the side walls, top wall, and the paddle.
9. Detect ball collisions with bricks, destroy hit bricks, and increase the score by 10 points per brick.
10. Display the "YOU WIN!" message when all bricks are destroyed or "GAME OVER" if the ball passes belowthe paddle.
11. Provide Restart Game and Exit buttons to reset the game state or close the application window.
12. Execute the game and verify paddle responsiveness, ball physics, brick collision, score tracking, win/game-over states, and restart functionality.

PROGRAM CODE

```
import tkinter as tk


# Game Settings WIDTH =
600
HEIGHT = 450
   PADDLE_WIDTH = 100
    PADDLE_HEIGHT = 12
BALL_SIZE = 16
SPEED = 15

class BrickBreaker:
        def __init__(self, root):
                self.root.title("BRICK BREAKER")
                self.root.resizable(False, False)
                self.root.configure(bg="#172554")


                tk.Label(
                root,
                text="BRICK BREAKER",
                font=("Arial Black", 22),
                bg="#172554",
                fg="white" ).pack(pady=8)


                self.score_label = tk.Label(
                        root,
                        text="SCORE: 0", font=
                        ("Arial", 15, "bold"),
                        bg="#172554",
                        fg="#FACC15"
                ) self.score_label.pack()


                self.canvas = tk.Canvas(
                        root,
                        width=WIDTH,
                        height=HEIGHT,
                        bg="#0F172A",
                        highlightthickness=2,
                        highlightbackground="#38BDF8"
                )
                self.canvas.pack(padx=15, pady=10)

                buttons = tk.Frame(root, bg="#172554")
                buttons.pack(pady=8)

                tk.Button(
                        buttons,
                        text="RESTART GAME",
                        font=("Arial", 11, "bold"),
                        bg="#22C55E", fg="black",
                        command=self.restart

                ).pack(side="left", padx=8)

                tk.Button(
                        buttons,
                        text="EXIT", font=
                        ("Arial", 11, "bold"),
                        bg="#EF4444",
                        fg="white",
```

```
def restart(self):
        if self.timer is not None:
                self.root.after_cancel(self.timer)
                self.timer = None

        self.canvas.delete("all")
        self.score = 0
        self.running = True
        self.score_label.config(text="SCORE: 0")

        self.paddle = self.canvas.create_rectangle(
                WIDTH // 2 - PADDLE_WIDTH // 2, HEIGHT - 35,
                WIDTH // 2 + PADDLE_WIDTH // 2, HEIGHT - 35 + PADDLE_HEIGHT,
                fill="#38BDF8", outline=""
        )

        self.ball = self.canvas.create_oval(
                WIDTH // 2 - BALL_SIZE // 2, HEIGHT - 65,
                WIDTH // 2 + BALL_SIZE // 2, HEIGHT - 65 + BALL_SIZE,
                fill="#FACC15", outline=""
        )

        self.dx = 4
        self.dy = -4

        self.bricks = [] colors=["#F87171","#FB923C","#FACC15","#4ADE80",
        "#60A5FA"]

        for row in range(5):
                for col in range(10):
                x1=8+col*59
                y1=30+row*25
                brick = self.canvas.create_rectangle(
                                x1, y1, x1 + 53, y1 + 19,
                                fill=colors[row], outline="#0F172A"
                        ) self.bricks.append(brick)


        self.update_game()

def move_left(self, event=None):
        if not self.running:
                return
        x1, y1, x2, y2 = self.canvas.coords(self.paddle) if x1
        > 0:
                self.canvas.move(self.paddle, -25, 0)

def move_right(self, event=None):
        if not self.running:
                return
        x1, y1, x2, y2 = self.canvas.coords(self.paddle) if x2
        < WIDTH:
                self.canvas.move(self.paddle, 25, 0)
```

```
def update_game(self): if
        not self.running:
            return

    self.canvas.move(self.ball, self.dx, self.dy)
    bx1, byl, bx2, by2 = self.canvas.coords(self.ball)

    #Bounceofftheleftandrightwallsandthetop wall if
    bx1 <= 0 or bx2 >= WIDTH:
        self.dx = -self.dx
    if byl <= 0:
        self.dy = abs(self.dy)

    # Bounce off the paddle
    px1, pyl, px2, py2 = self.canvas.coords(self.paddle)
    if(bx2>=px1andbx1<=px2andby2>=pyland byl <= py2 and self.dy > 0):
        self.dy = -abs(self.dy)

    # Remove a brick when the ball touches it
    hit_brick = None
    for brick in self.bricks:
        if brick in self.canvas.find_all():
            x1, y1, x2, y2 = self.canvas.coords(brick)
            ifbx2>=x1andbx1<=x2andby2>=y1 and byl <= y2:
                hit_brick = brick
                break

    if hit_brick is not None:
        self.canvas.delete(hit_brick) self.bricks.remove(hit_brick)
        self.dy = -self.dy
        self.score += 10 self.score_label.config(text=f"SCORE: {self.score}")



    if not self.bricks:
        self.finish("YOU WIN!")
        return

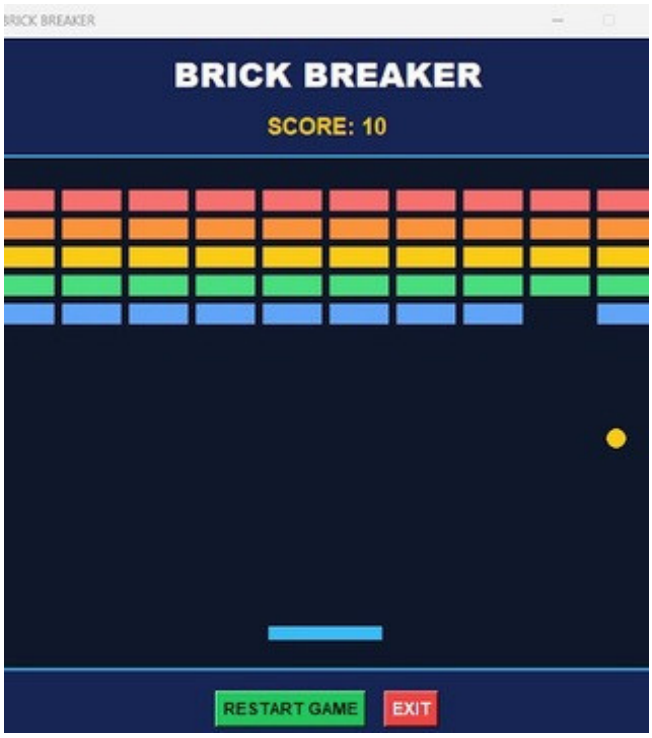
    if by2 >= HEIGHT:
        self.finish("GAME OVER")
        return

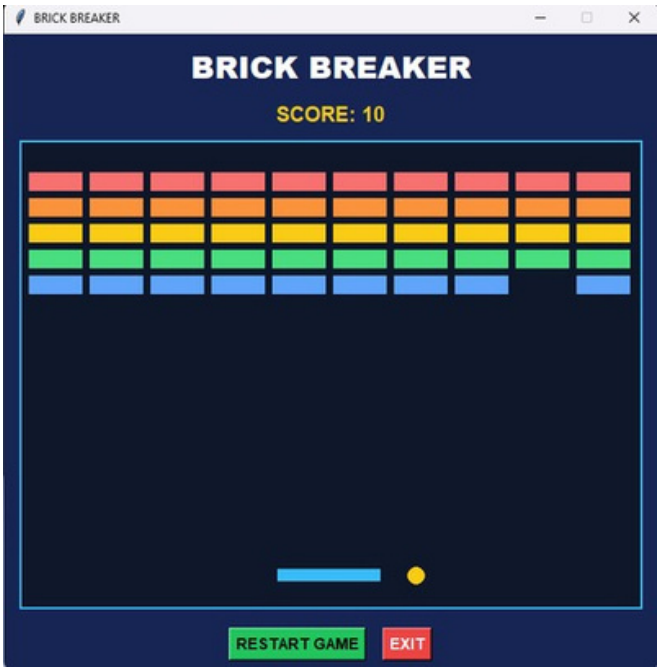
    self.timer = self.root.after(SPEED, self.update_game)
```

```
        def finish(self, message):
        self.running = False
        self.timer = None
        self.canvas.create_rectangle(
                        140, 165, 460, 275,
                        fill="#172554", outline="#38BDF8", width=2
                )
                self.canvas.create_text(
                        WIDTH // 2, 200, text=message,
                        font=("Arial Black", 25),
                        fill="#FACC15" if message == "YOU WIN!" else "#EF4444"
                )
                self.canvas.create_text(
                WIDTH // 2, 238,
                text=f"Final Score: {self.score}",
                font=("Arial", 17, "bold"), fill="white"
                )

# Main Application
if __name__ == "__main__":
        window = tk.Tk()
        game = BrickBreaker(window)
        window.mainloop()
```

OUTPUT





RESULT

Thus, an interactive Brick Breaker game was developed successfully using Python Tkinter. The game includes a graphical user interface, paddle movement, ball animation, brick collision detection, score calculation, win and game-over conditions, restart functionality, and exit controls. The project demonstrates the use of Python programming, GUI design, event handling, collision detection, and basic game development concepts.